PBD – Lista projektowa nr 3 – raport

Środowisko: PostgreSQL 17 oraz Oracle Database 23ai Free (`gvenzl/oracle-free`) w Dockerze (`docker-compose.yml`), dane z listy 1.

- **PostgreSQL** – zadania 1–10 (`postgres/zadania_psql.sql`, wynik: `postgres/wynik.txt`).
- **Oracle** – zadania 4–10 (wymagane w obu SZBD) oraz 11–13 (tylko Oracle) (`oracle/zadania_oracle.sql`, wynik: `oracle/wynik.txt`).

Zadania 1–6 są bez podzapytań; w zad. 7–10 podzapytania są dozwolone. Wyniki we wszystkich zadaniach zgadzają się z przykładami z listy (poza kolejnością wierszy tam, gdzie nie była wymagana).

Część 1 (bez podzapytań, PostgreSQL)

Zad. 1

Zbiór wierszy taki sam jak w przykładzie. Dosłownie czytany warunek „posiadające wroga na A” wykluczałby koty bez incydentów, a przykład je zawiera (LUCEK, MICKA, PUCEK). Dlatego warunek wroga na A dotyczy tylko gałęzi „przed szefem”: `bez incydentu OR (przed szefem AND wróg na A)`. `LEFT JOIN` do `Wrogowie_kocurow` zostawia koty bez incydentów (`wk.pseudo IS NULL`), a `LEFT JOIN` do szefa daje `NULL` dla TYGRYSA, który przez to nie spełnia drugiej gałęzi. Imię wroga jest już w `Wrogowie_kocurow`, więc tabela `Wrogowie` nie jest potrzebna. JACKA dołączam warunkiem w `ON`, żeby porównać daty bez podzapytania. `DISTINCT` usuwa powtórzenia kotów z kilkoma incydentami.

```
SELECT DISTINCT k.imie "W stadzie przed szefem lub bez incydentu",
       CASE WHEN k.w_stadku_od < j.w_stadku_od THEN 'TAK' ELSE 'NIE' END "Przed JACKIEM"
FROM Kocury k
LEFT JOIN Kocury s ON k.szef = s.pseudo
LEFT JOIN Wrogowie_kocurow wk ON wk.pseudo = k.pseudo
JOIN Kocury j ON j.imie = 'JACEK'
WHERE wk.pseudo IS NULL OR (k.w_stadku_od < s.w_stadku_od AND wk.imie_wroga LIKE '%A')
ORDER BY 1;
```

```
 W stadzie przed szefem lub bez incydentu | Przed JACKIEM
------------------------------------------+---------------
 LUCEK                                    | NIE
 MICKA                                    | NIE
 PUCEK                                    | TAK
 ZUZIA                                    | TAK
(4 rows)
```

Zad. 2

`FULL JOIN` tabeli `Kocury` samej ze sobą: wiersze niedopasowane to brak podwładnego lub przełożonego (`COALESCE` wstawia napis). Filtr płci jest w `WHERE`, a `COALESCE(plec, 'M')` przepuszcza strony uzupełnione `NULL`-ami. Gdyby filtr był w `ON`, wróciłyby koty płci żeńskiej. Skutek uboczny: ZOMBI, który ma tylko podwładne kotki, nie ma wiersza „Brak podwladnego” — dokładnie tak jak w przykładzie.

```
SELECT COALESCE(s.pseudo, 'Brak przelozonego') "Przelozony",
       COALESCE(p.pseudo, 'Brak podwladnego') "Podwladny"
FROM Kocury p
FULL JOIN Kocury s ON p.szef = s.pseudo
WHERE COALESCE(p.plec, 'M') = 'M' AND COALESCE(s.plec, 'M') = 'M'
ORDER BY 1, 2;
```

```
    Przelozony     |    Podwladny
-------------------+------------------
 BOLEK             | Brak podwladnego
 Brak przelozonego | TYGRYS
 LYSY              | PLACEK
 LYSY              | RURA
 MALY              | Brak podwladnego
 MAN               | Brak podwladnego
 PLACEK            | Brak podwladnego
 RAFA              | MALY
 RAFA              | MAN
 RURA              | Brak podwladnego
 TYGRYS            | BOLEK
 TYGRYS            | LYSY
 TYGRYS            | RAFA
 TYGRYS            | ZOMBI
 ZERO              | Brak podwladnego
(15 rows)
```

Zad. 3

Na polu mogą polować koty z bandy o terenie `POLE` oraz `CALOSC` (cała okolica obejmuje pole). Suma przydziałów w bandzie dotyczy **całej** bandy (200 = 103+50+22+25), a nie tylko kotów z wrogami, więc trzeba dołączyć kolegów z bandy (`m`). To mnoży wiersze przez liczbę wrogów kota, dlatego sumę dzielę przez `COUNT(DISTINCT wrog)`. Procent = `ROUND(100 * przydzial / suma)`.

```
SELECT k.pseudo "PSEUDO", k.przydzial_myszy "PRZYDZIAL_MYSZY",
       ROUND(SUM(m.przydzial_myszy) / COUNT(DISTINCT w.imie_wroga)) "SUM_W_BANDZIE",
       ROUND(100 * k.przydzial_myszy * COUNT(DISTINCT w.imie_wroga) / SUM(m.przydzial_myszy))
"PROC_W_BANDZIE"
FROM Kocury k
JOIN Bandy b ON k.nr_bandy = b.nr_bandy
JOIN Wrogowie_kocurow wk ON wk.pseudo = k.pseudo
JOIN Wrogowie w ON w.imie_wroga = wk.imie_wroga
JOIN Kocury m ON m.nr_bandy = k.nr_bandy
WHERE b.teren IN ('POLE', 'CALOSC') AND w.stopien_wrogosci BETWEEN 6 AND 10
GROUP BY k.pseudo, k.przydzial_myszy, k.nr_bandy
ORDER BY k.nr_bandy, k.przydzial_myszy;
```

```
 PSEUDO | PRZYDZIAL_MYSZY | SUM_W_BANDZIE | PROC_W_BANDZIE
--------+-----------------+---------------+----------------
 BOLEK  |              50 |           200 |             25
 TYGRYS |             103 |           200 |             52
 LASKA  |              24 |           284 |              8
 RURA   |              56 |           284 |             20
(4 rows)
```

Część 2 (bez podzapytań, PostgreSQL i Oracle)

Składnia zapytań w obu SZBD jest taka sama; wyniki identyczne (pokazuję wynik z PostgreSQL, pełny wynik Oracle w `oracle/wynik.txt`).

Zad. 4

Liczę `COUNT(DISTINCT pseudo)`, bo kot z dwoma wrogami występuje w dwóch wierszach. Banda bez kotów (ROCKERSI) odpada przez zwykłe `JOIN`. Wynik w Oracle identyczny.

```
SELECT b.nazwa "Nazwa bandy", COUNT(DISTINCT k.pseudo) "Koty z wrogami"
FROM Bandy b
JOIN Kocury k ON k.nr_bandy = b.nr_bandy
JOIN Wrogowie_kocurow wk ON wk.pseudo = k.pseudo
GROUP BY b.nr_bandy, b.nazwa
ORDER BY b.nr_bandy;
```

```
   Nazwa bandy   | Koty z wrogami
-----------------+---------------
 SZEFOSTWO       |              3
 CZARNI RYCERZE  |              5
 BIALI LOWCY     |              3
 LACIACI MYSLIWI |              4
(4 rows)
```

Zad. 5

Drugi warunek („wróg gatunku CZLOWIEK i PIES”) to `COUNT(DISTINCT ...) = 2` po tych dwóch gatunkach. Logicznie zawiera się w pierwszym (kot z wrogami dwóch gatunków ma co najmniej dwóch wrogów), więc oba dają ten sam zbiór kotów. Oracle: ten sam zapis i wynik.

```
SELECT k.funkcja "Funkcja", k.pseudo "Pseudonim kota", COUNT(*) "Liczba wrogow"
FROM Kocury k
JOIN Wrogowie_kocurow wk ON wk.pseudo = k.pseudo
JOIN Wrogowie w ON w.imie_wroga = wk.imie_wroga
GROUP BY k.pseudo, k.funkcja
HAVING COUNT(*) > 1
    OR COUNT(DISTINCT CASE WHEN w.gatunek IN ('CZLOWIEK', 'PIES') THEN w.gatunek END) = 2
ORDER BY k.pseudo;
```

```
 Funkcja  | Pseudonim kota | Liczba wrogow
----------+----------------+---------------
 DZIELCZY | BOLEK          |             2
 MILUSIA  | LASKA          |             2
 SZEFUNIO | TYGRYS         |             2
(3 rows)
```

Zad. 6

`MINUS` w Oracle, `EXCEPT` w PostgreSQL. Operator zbiorowy wymaga tych samych kolumn po obu stronach, więc odjęcie jest od bandy złączonej z kotami. Wynik taki sam w obu SZBD.

a) bez podzapytań i operatorów zbiorowych

```
SELECT b.nr_bandy "NR BANDY", b.nazwa "NAZWA", b.teren "TEREN"
FROM Bandy b
LEFT JOIN Kocury k ON k.nr_bandy = b.nr_bandy
WHERE k.pseudo IS NULL;
```

```
 NR BANDY |  NAZWA   |  TEREN
----------+----------+---------
        5 | ROCKERSI | ZAGRODA
(1 row)
```

b) operator zbiorowy (PostgreSQL `EXCEPT`; w Oracle `MINUS`)

```
SELECT nr_bandy "NR BANDY", nazwa "NAZWA", teren "TEREN" FROM Bandy
EXCEPT
SELECT b.nr_bandy, b.nazwa, b.teren FROM Bandy b JOIN Kocury k ON k.nr_bandy = b.nr_bandy;
```

```
 NR BANDY |  NAZWA   |  TEREN
----------+----------+---------
        5 | ROCKERSI | ZAGRODA
(1 row)
```

Część 3 (z podzapytaniami, PostgreSQL i Oracle)

Wyniki w obu SZBD identyczne; pokazuję PostgreSQL. Różnice składniowe: `NVL` (Oracle) vs `COALESCE`, `NLS_DATE_LANGUAGE` w `TO_CHAR` (Oracle).

Zad. 7

`NOT EXISTS` dla szaraków (nie istnieje kot z tej samej bandy z mniejszym przydziałem — remisy, np. UCHO i MALY, zostają). Prominenci: przydział > średnia (51,67) ze wszystkich kotów.

```
SELECT pseudo "Pseudonim", przydzial_myszy "Zjada", nr_bandy "Banda", 'Prominent' "Typ",
       CASE WHEN przydzial_myszy > 60 THEN 'powyzej 60' ELSE 'ponizej 60' END "Ocena"
FROM Kocury
WHERE przydzial_myszy > (SELECT AVG(przydzial_myszy) FROM Kocury)
UNION ALL
SELECT k.pseudo, k.przydzial_myszy, k.nr_bandy, 'Szarak',
       CASE WHEN k.przydzial_myszy > 60 THEN 'powyzej 60' ELSE 'ponizej 60' END
FROM Kocury k
WHERE NOT EXISTS (SELECT 1 FROM Kocury m WHERE m.nr_bandy = k.nr_bandy AND m.przydzial_myszy <
k.przydzial_myszy)
ORDER BY "Typ", "Banda", "Zjada" DESC;
```

```
 Pseudonim | Zjada | Banda |    Typ    |   Ocena
-----------+-------+-------+-----------+------------
 TYGRYS    |   103 |     1 | Prominent | powyzej 60
 LYSY      |    72 |     2 | Prominent | powyzej 60
 PLACEK    |    67 |     2 | Prominent | powyzej 60
 SZYBKA    |    65 |     2 | Prominent | powyzej 60
 RURA      |    56 |     2 | Prominent | ponizej 60
 ZOMBI     |    75 |     3 | Prominent | powyzej 60
 KURKA     |    61 |     3 | Prominent | powyzej 60
 RAFA      |    65 |     4 | Prominent | powyzej 60
 MALA      |    22 |     1 | Szarak    | ponizej 60
 LASKA     |    24 |     2 | Szarak    | ponizej 60
 PUSZYSTA  |    20 |     3 | Szarak    | ponizej 60
 UCHO      |    40 |     4 | Szarak    | ponizej 60
 MALY      |    40 |     4 | Szarak    | ponizej 60
(13 rows)
```

Koty, które nie są niczyim szefem — `NOT EXISTS`

```
SELECT k.pseudo "Nie sa niczyim szefem"
FROM Kocury k
WHERE NOT EXISTS (SELECT 1 FROM Kocury p WHERE p.szef = k.pseudo)
ORDER BY 1;
```

```
 Nie sa niczyim szefem
----------------------
 BOLEK
 DAMA
 LASKA
 LOLA
 MALA
 MALY
 MAN
 PLACEK
 PUSZYSTA
 RURA
 SZYBKA
 UCHO
 ZERO
(13 rows)
```

Koty, które nie są niczyim szefem — `NOT IN`

`NOT IN` zwraca 0 wierszy. Kolumna `szef` zawiera `NULL` (TYGRYS nie ma szefa). `x NOT IN (a, b, NULL)` to `x <> a AND x <> b AND x <> NULL`, a ostatni człon jest `UNKNOWN`, więc cały warunek nigdy nie jest `TRUE`. `NOT EXISTS` porównuje przez korelację `p.szef = k.pseudo`; wiersz z `NULL` po prostu nic nie pasuje i nie psuje wyniku, dlatego dostajemy poprawne 13 kotów. Poprawka dla `NOT IN`: `WHERE szef IS NOT NULL` w podzapytaniu.

```
SELECT pseudo "Nie sa niczyim szefem"
FROM Kocury
WHERE pseudo NOT IN (SELECT szef FROM Kocury)
ORDER BY 1;
```

```
 Nie sa niczyim szefem
----------------------
(0 rows)
```

Zad. 8

Rzeczywisty przydział = `przydzial_myszy + COALESCE(myszy_extra, 0)`; bez obsługi `NULL` suma byłaby `NULL` dla większości kotów. Średnia w kolumnie wymaga podzapytania skalarnego (w `HAVING` i w `SELECT`). W Oracle `NVL` zamiast `COALESCE`.

Z wyświetleniem średniej

```
SELECT nr_bandy "Lepsze bandy",
       ROUND(AVG(przydzial_myszy + COALESCE(myszy_extra, 0)), 1) "Sredni przydzial w bandzie",
       (SELECT ROUND(AVG(przydzial_myszy + COALESCE(myszy_extra, 0)), 1) FROM Kocury) "[Sredni
przydzial]"
FROM Kocury
GROUP BY nr_bandy
HAVING AVG(przydzial_myszy + COALESCE(myszy_extra, 0))
     > (SELECT AVG(przydzial_myszy + COALESCE(myszy_extra, 0)) FROM Kocury)
ORDER BY nr_bandy;
```

```
 Lepsze bandy | Sredni przydzial w bandzie | [Sredni przydzial]
--------------+----------------------------+--------------------
            1 |                       80.5 |               63.8
            2 |                       66.6 |               63.8
(2 rows)
```

Bez wyświetlania średniej

```
SELECT nr_bandy "Lepsze bandy",
       ROUND(AVG(przydzial_myszy + COALESCE(myszy_extra, 0)), 1) "Sredni przydzial w bandzie"
FROM Kocury
GROUP BY nr_bandy
HAVING AVG(przydzial_myszy + COALESCE(myszy_extra, 0))
     > (SELECT AVG(przydzial_myszy + COALESCE(myszy_extra, 0)) FROM Kocury)
ORDER BY nr_bandy;
```

```
 Lepsze bandy | Sredni przydzial w bandzie
--------------+----------------------------
            1 |                       80.5
            2 |                       66.6
(2 rows)
```

Zad. 9

Pełna nazwa miesiąca: `FMMONTH` (`FM` usuwa wypełnienie spacjami). Oracle wymaga jawnego `NLS_DATE_LANGUAGE=ENGLISH`, bo inaczej nazwy zależą od ustawień sesji. Sortowanie po numerze miesiąca (`EXTRACT`), który musi być też w `GROUP BY`. Miesiące bez rekrutów (kwiecień, czerwiec) nie występują.

```
SELECT TO_CHAR(w_stadku_od, 'FMMONTH') "Miesiac", COUNT(*) "Liczba rekrutow"
FROM Kocury
GROUP BY EXTRACT(MONTH FROM w_stadku_od), TO_CHAR(w_stadku_od, 'FMMONTH')
ORDER BY EXTRACT(MONTH FROM w_stadku_od);
```

```
  Miesiac  | Liczba rekrutow
-----------+-----------------
 JANUARY   |               3
 FEBRUARY  |               1
 MARCH     |               2
 MAY       |               2
 JULY      |               2
 AUGUST    |               1
 SEPTEMBER |               2
 OCTOBER   |               2
 NOVEMBER  |               2
 DECEMBER  |               1
(10 rows)
```

Zad. 10

Bez `MAX`: `>= ALL` na potrojonych przydziałach. „Miluśki operujące w sadzie” to miluśki z bandy o terenie `SAD` oraz `CALOSC` (jak w zad. 3); MICKA z bandy 1 ma 25, więc próg wynosi 75. Biorąc tylko `SAD`, próg byłby 60 i wynik nie zgadzałby się z przykładem. Uwaga na `NULL`: gdyby któraś miluśka miała `przydzial_myszy = NULL`, `>= ALL` nie zwróciłby żadnego wiersza (ten sam mechanizm co `NOT IN` w zad. 7); w danych taki przypadek nie występuje.

```
SELECT imie "IMIE", funkcja "FUNKCJA", przydzial_myszy "PRZYDZIAL MYSZY"
FROM Kocury
WHERE przydzial_myszy >= ALL (SELECT 3 * m.przydzial_myszy
                              FROM Kocury m JOIN Bandy b ON m.nr_bandy = b.nr_bandy
                              WHERE m.funkcja = 'MILUSIA' AND b.teren IN ('SAD', 'CALOSC'));
```

```
  IMIE   | FUNKCJA  | PRZYDZIAL MYSZY
---------+----------+-----------------
 MRUCZEK | SZEFUNIO |             103
 KOREK   | BANDZIOR |              75
(2 rows)
```

Część 4 (tylko Oracle)

Zad. 11

`START WITH` na męskich bandziorach, w `WHERE` filtr płci: przejście drzewa idzie też przez kotkę (KURKA), żeby dojść do ZERO na poziomie 3, ale same kotki nie są wyświetlane.

```
SELECT LEVEL "Poziom", pseudo "Pseudonim", funkcja "Funkcja", nr_bandy "Nr bandy"
FROM Kocury
WHERE plec = 'M'
START WITH plec = 'M' AND funkcja = 'BANDZIOR'
CONNECT BY PRIOR pseudo = szef
ORDER SIBLINGS BY pseudo;
```

```
    Poziom Pseudonim       Funkcja      Nr bandy
---------- --------------- ---------- ----------
         1 LYSY            BANDZIOR            2
         2 PLACEK          LOWCZY              2
         2 RURA            LAPACZ              2
         1 ZOMBI           BANDZIOR            3
         3 ZERO            KOT                 3
```

Zad. 12

Drzewo budowane od dołu (`CONNECT BY PRIOR szef = pseudo`) od kotów spełniających warunki startowe; wcięcie `LPAD(' ', 4*(LEVEL-1))`. Staż > 15 lat: `MONTHS_BETWEEN(DATE '2024-07-17', w_stadku_od) > 180`. Brak dodatkowego przydziału: `NVL(myszy_extra, 0) = 0`.

```
SELECT LPAD(' ', 4 * (LEVEL - 1)) || pseudo "Droga sluzbowa"
FROM Kocury
START WITH plec = 'M' AND NVL(myszy_extra, 0) = 0
       AND MONTHS_BETWEEN(DATE '2024-07-17', w_stadku_od) > 180
CONNECT BY PRIOR szef = pseudo
ORDER SIBLINGS BY pseudo;
```

```
Droga sluzbowa
------------------------------
BOLEK
    TYGRYS
MAN
    RAFA
        TYGRYS
PLACEK
    LYSY
        TYGRYS
RAFA
    TYGRYS
```

Zad. 13

Trzy `LEFT JOIN` po szefie (maksymalna głębokość w danych to 3). Kolejność wierszy w Oracle bez `ORDER BY` jest dowolna.

a) same złączenia

```
SELECT k.imie "Imie", k.funkcja "Funkcja", s1.imie "Szef 1", s2.imie "Szef 2", s3.imie "Szef 3"
FROM Kocury k
LEFT JOIN Kocury s1 ON k.szef = s1.pseudo
LEFT JOIN Kocury s2 ON s1.szef = s2.pseudo
LEFT JOIN Kocury s3 ON s2.szef = s3.pseudo
WHERE k.funkcja IN ('KOT', 'MILUSIA');
```

```
Imie            Funkcja    Szef 1          Szef 2          Szef 3
--------------- ---------- --------------- --------------- ---------------
LUCEK           KOT        PUNIA           KOREK           MRUCZEK
SONIA           MILUSIA    KOREK           MRUCZEK
BELA            MILUSIA    BOLEK           MRUCZEK
LATKA           KOT        PUCEK           MRUCZEK
DUDEK           KOT        PUCEK           MRUCZEK
MICKA           MILUSIA    MRUCZEK
RUDA            MILUSIA    MRUCZEK
```

b) drzewo, `SYS_CONNECT_BY_PATH`, `CONNECT_BY_ROOT`

Drzewo budowane w górę (`CONNECT BY PRIOR szef = pseudo`). `CONNECT_BY_ROOT` daje imię i funkcję kota startowego, a `CONNECT_BY_ISLEAF = 1` zostawia jeden wiersz na kota — ten na szczycie hierarchii, z pełną ścieżką. `SYS_CONNECT_BY_PATH` zaczyna ścieżkę od samego kota, więc odcinam ją `SUBSTR` o długość jego imienia plus separator.

```
SELECT CONNECT_BY_ROOT imie "Imie", CONNECT_BY_ROOT funkcja "Funkcja",
       SUBSTR(SYS_CONNECT_BY_PATH(imie, ' | '), LENGTH(CONNECT_BY_ROOT imie) + 4) "Imiona
kolejnych szefow"
FROM Kocury
WHERE CONNECT_BY_ISLEAF = 1
START WITH funkcja IN ('KOT', 'MILUSIA')
CONNECT BY PRIOR szef = pseudo;
```

```
Imie            Funkcja    Imiona kolejnych szefow
--------------- ---------- ----------------------------------------------------------------
BELA            MILUSIA     | BOLEK | MRUCZEK
MICKA           MILUSIA     | MRUCZEK
RUDA            MILUSIA     | MRUCZEK
DUDEK           KOT         | PUCEK | MRUCZEK
SONIA           MILUSIA     | KOREK | MRUCZEK
LATKA           KOT         | PUCEK | MRUCZEK
LUCEK           KOT         | PUNIA | KOREK | MRUCZEK
```

Podsumowanie

Wszystkie 13 zadań wykonane i sprawdzone na działających bazach: zad. 1–3 w PostgreSQL, 4–10 w PostgreSQL i Oracle, 11–13 w Oracle. Wyniki odpowiadają przykładom z listy.

Najważniejsze wnioski:

- **`NULL` w `NOT IN` / `ALL`** – `NOT IN` z podzapytaniem zwracającym `NULL` (kolumna `szef`) nie zwraca nic, a `NOT EXISTS` działa poprawnie (zad. 7); to samo dotyczy `>= ALL` (zad. 10). Podobnie `myszy_extra` wymaga `COALESCE` / `NVL` przy sumowaniu (zad. 8, 12).
- **Interpretacja treści** – „polowanie na polu / w sadzie” obejmuje też bandę z terenem `CALOSC` (zad. 3, 10); tylko tak wychodzą wyniki z przykładów.
- **Złączenia a agregaty** – dołączenie tabeli „mnoży” wiersze, więc sumy po takim złączeniu trzeba skorygować (zad. 3: dzielenie przez `COUNT(DISTINCT ...)`).

- **Hierarchie** – `CONNECT BY` w Oracle pozwala w jednym `SELECT` budować drzewo w obu kierunkach (zad. 11–13); odpowiednik w PostgreSQL wymagałby rekurencyjnego CTE.
- **Różnice PostgreSQL / Oracle** – `EXCEPT` / `MINUS`, `COALESCE` / `NVL`, wymuszenie języka nazw miesięcy w Oracle (zad. 9).